Seminar 4

Object-Oriented Design, IV1350

Filip Kehl, fkehl@kth.se

2026-05-31

Project Members:

Filip Kehl, fkehl@kth.se

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.
It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1 Introduction

This report describes the seminar 4 extensions to the Repair Electric Bike program: exception handling for the two alternative flows highlighted by the task description (task 1) and three GoF design patterns (task 2). The program from seminar 3 is the starting point, the seminar 2 design is followed wherever it has not been changed for the better.

The task requires the following:

- Task 1a: throw an exception when a customer is searched for by phone number and no such customer is in the registry.
- Task 1b: throw an exception when the integration layer cannot reach the database; simulated by always throwing for one hard-coded phone number.
- All nine exception-handling best practices from chapter 8 of the textbook (covered in section 4.1 of this report).
- Task 2a: an Observer pattern with at least two observer implementations (one to System.out and one to a file).
- Task 2b: two more GoF patterns beyond Observer and Template Method.
- Unit tests covering the new behaviour.
- A printout of a sample run, plus the usual IMRaD chapters.

2 Method

2.1 Exception handling

The exceptions were designed before they were written. The first decision was checked vs. unchecked. CustomerNotFoundException is checked because not finding a customer is a predictable, recoverable business condition that the calling layer can handle (the receptionist tries another number). DatabaseFailureException is unchecked because a failure of the integration layer to reach the database is a technical problem that the use-case code cannot meaningfully recover from in the normal flow. The program must report it and stop the current business action. Both choices are stated in the Javadoc of the exception classes and verified by CustomerNotFoundExceptionTest and DatabaseFailureExceptionTest.

Each exception was then named after its error condition ("CustomerNotFound" and "DatabaseFailure"), given a Javadoc comment and equipped with a field carrying the identifier that triggered it (the missing phone number, or the failing identifier) so that the handler can act on it. The messages passed to super(message) read as complete sentences. DatabaseFailureException also takes an optional throwable cause so that an underlying technical exception can be chained inside it without leaking through the public interface.

Exception safety was enforced by checking parameters at the start of state-changing methods. The example is RepairOrder.setDiscountStrategy(null), which throws IllegalArgumentException before touching the field. The test testSettingNullDiscountStrategyThrowsAndKeepsState verifies the strategy field is unchanged after the throw.

User notification and developer notification were split: the View prints a short, non-technical message to System.out for both exception paths, and additionally writes a stack trace to repair-bike-error-log.txt through ErrorFileLogger for the DatabaseFailureException. CustomerNotFoundException is a business condition and is not logged, since the task is explicit about only logging failures that indicate the program is not functioning as intended.

2.2 Observer pattern (task 2a)

RepairOrder was made the Subject of the Observer pattern. The reasoning is that the order is the entity whose updates need to be broadcast, and it is the only place that knows when a state transition has happened. Exposing those events through any other class would force that class to either poll the order or be told about the change by the order, both of which collapse back into the same observer notification.

The data pushed to the observers is an immutable RepairOrderDTO. This DTO is also the type returned by every controller method, so the controller-level snapshot and the observer-level snapshot are the same definition. Using one DTO type keeps the public interface small and means that observers never have to call back into the controller or the model to fetch additional information, they get everything they need from the supplied DTO. This addresses the assessment criteria about which data is passed from the observed object to the observer.

The two concrete observers are RepairOrderView and RepairOrderLogger. Both implement the same RepairOrderObserver interface. Neither calls back into the controller or the model, they only react to what is pushed to them. The reference from observers to the subject is set up at startup: Main instantiates both observers and registers them on the Controller, which then attaches them to every RepairOrder it creates. This keeps the Controller's compile-time knowledge limited to the RepairOrderObserver interface and lets observers be added or removed without touching the controller's code.

2.3 Singleton (task 2b, first pattern)

RegistryCreator is the Singleton. The motivation is that the customer database and the repair-order database are shared resources, so the program needs a single pair of registry instances. Multiple RegistryCreator instances would each have their own in-memory state and produce silently inconsistent behaviour. The implementation uses double-checked locking with a volatile field.

The well-known cost of the Singleton pattern is that unit tests cannot easily have a clean state per test. To compensate, the class exposes a static factory createForTests() that returns a fresh, non-singleton instance. Production code never calls this factory, the unit tests for the Controller and the View call it in their @BeforeEach and so get a clean registry pair every time. This is the pragmatic mitigation for the testability issue without weakening the production guarantee that getInstance() returns the instance.

2.4 Strategy (task 2b, second pattern)

The motivation for the Strategy pattern is that the workshop may want to apply different discount rules over time without modifying RepairOrder. The DiscountStrategy interface defines the algorithm, three concrete implementations are provided: NoDiscount (the default), LoyaltyDiscount (a fixed percentage off for returning customers), and WinterDiscount (a smaller percentage off between December and February). RepairOrder holds a single DiscountStrategy field and delegates to it from getDiscount(). Setting a new strategy through setDiscountStrategy also notifies all observers, so any displayed view picks up the new total. WinterDiscount takes a Supplier<LocalDate> for tests, so the discount can be checked for arbitrary months without manipulating the system.

2.5 Relation to seminar 2 and 3

The seminar 3 review surfaced three regressions versus the seminar 2 design: the Controller returned the RepairOrder entity instead of a DTO, the Printer depended on RepairOrder, and the Controller did the printing instead of letting the order print itself. All three were carried forward into the seminar 4 implementation and are fixed here: Controller methods return a RepairOrderDTO,

Printer.printRepairOrder takes a String, RepairOrder.accept(Printer) sets its own state to ACCEPTED, notifies observers, and asks the printer to print. These changes recover the encapsulation level of the seminar 2 design while still satisfying the seminar 4 task.

2.6 Unit tests

Tests were written for every new class and every new behaviour. The exception classes have dedicated tests (CustomerNotFoundExceptionTest, DatabaseFailureExceptionTest) that verify the carried payload, the message, the cause chain, and the checked-vs-unchecked classification. CustomerRegistryTest verifies the two exception paths through the integration layer. ControllerTest does the same at the controller level and uses RegistryCreator.createForTests() to keep the singleton state out of the test fixtures. The Observer pattern is covered by RepairOrderObserverTest with a RecordingObserver stub that counts how many notifications each observer receives. The test cases include add, remove, defensive iteration on observer failure, and the discount-strategy change notification. RepairOrderViewTest and RepairOrderLoggerTest wire a real RepairOrder to each concrete observer and verify the output. ViewTest captures System.out and uses an ErrorFileLogger pointed at a temporary file to verify that the user message is shown for both exception paths and that the developer log is written for the database failure but not for the missing customer.

3 Result

The complete source code is available in the public Git repository:
https://github.com/fikehl/iv1350-repair-electric-bike-sem4-

3.1 New and changed classes

- integration/CustomerNotFoundException (new, checked) — thrown by CustomerRegistry when the supplied phone number is not in the registry. Carries the missing phone number.
- integration/DatabaseFailureException (new, unchecked) — thrown by CustomerRegistry to simulate an unreachable database. Carries the failing identifier and accepts an optional Throwable cause.
- integration/CustomerRegistry — updated to throw the two exceptions above. The database failure is simulated for the hardcoded phone number 0700000000.
- integration/RegistryCreator — now a Singleton with getInstance() and a test-only createForTests() factory.
- integration/Printer — now takes a String instead of a RepairOrder, so the integration layer no longer depends on the model entity.
- util/ErrorFileLogger (new) — appends a timestamped stack trace to repair-bike-error-log.txt. Inspired by the textbook's FileLogger (listing 9.1) but not copied from it.
- model/RepairOrderObserver (new) — the Observer role interface. Single method repairOrderUpdated(RepairOrderDTO).
- model/RepairOrderDTO (new) — immutable snapshot returned from every Controller method and pushed to every observer.
- model/DiscountStrategy + NoDiscount, LoyaltyDiscount, WinterDiscount (new) — the Strategy role and three concrete implementations.
- model/RepairOrder — now the Subject of the Observer pattern, holds a DiscountStrategy, has accept(Printer) and toDTO(), and no longer exposes its internal customer/problem/diagnostic fields as public getters.
- controller/Controller — methods return RepairOrderDTO instead of RepairOrder; has addRepairOrderObserver and removeRepairOrderObserver and applyDiscountStrategy.
- view/View — wraps the calls to Controller.searchCustomer in try/catch and uses the ErrorFileLogger for the technical failure.
- view/RepairOrderView, RepairOrderLogger (new) — the two concrete observers.

3.2 Sample run

The full output of one execution is shown below. The execution covers (1) the basic flow with a loyalty discount applied, (2) the alternative flow where the phone number is unknown to the customer registry, and (3) the simulated database failure for the hardcoded phone number 0700000000. The contents of repair-bike-error-log.txt produced by the run are included at the end.

```
########################################################################

RECEPTIONIST: Customer arrives. Searching for customer with phone 0701112233.
System returned: Customer[name=Anna Andersson, phone=0701112233, email=anna@example.com,
Bike[brand=Crescent, model=Elina E8, serialNumber=CR-E8-00417]]

########################################################################
```

```
RECEPTIONIST: Customer describes the problem. Registering it.
System returned a new repair order:
==========================================
           REPAIR ORDER #1
==========================================
Date:    2026-05-16 21:00
State:   NEWLY_CREATED
------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
------------------------------------------
No diagnostic report has been added yet.
Estimated completion: 2026-05-23 21:00
==========================================



##########################################################################

TECHNICIAN: Performing diagnostic and proposing repair tasks.

>>> RepairOrderView: repair order #1 updated (state = READY_FOR_APPROVAL)
==========================================
           REPAIR ORDER #1
==========================================
Date:    2026-05-16 21:00
State:   READY_FOR_APPROVAL
------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.

Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK

Subtotal:    2000 SEK
Discount:    0 SEK (No discount)
Total cost:  2000 SEK
------------------------------------------
Estimated completion: 2026-05-23 21:00
==========================================

System returned the updated repair order:
==========================================
           REPAIR ORDER #1
==========================================
Date:    2026-05-16 21:00
State:   READY_FOR_APPROVAL
------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
------------------------------------------
Customer's problem description:
```

```
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.


Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK


Subtotal:    2000 SEK
Discount:    0 SEK (No discount)
Total cost:  2000 SEK
------------------------------------------
Estimated completion: 2026-05-23 21:00
==========================================



##########################################################################

RECEPTIONIST: Anna is a returning customer; applying loyalty discount.

>>> RepairOrderView: repair order #1 updated (state = READY_FOR_APPROVAL)
==========================================
           REPAIR ORDER #1
==========================================
Date:    2026-05-16 21:00
State:   READY_FOR_APPROVAL
------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.


Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK


Subtotal:    2000 SEK
Discount:    200 SEK (10% loyalty discount for returning customers)
Total cost:  1800 SEK
------------------------------------------
Estimated completion: 2026-05-23 21:00
==========================================



##########################################################################

RECEPTIONIST: Customer accepts the proposed repair tasks. Registering acceptance.

>>> RepairOrderView: repair order #1 updated (state = ACCEPTED)
==========================================
           REPAIR ORDER #1
==========================================
Date:    2026-05-16 21:00
State:   ACCEPTED
------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
```

```
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
-------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
-------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.

Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK

Subtotal:    2000 SEK
Discount:    200 SEK (10% loyalty discount for returning customers)
Total cost:  1800 SEK
-------------------------------------------
Estimated completion: 2026-05-23 21:00
===========================================

===========================================
           REPAIR ORDER #1
===========================================
Date:    2026-05-16 21:00
State:   ACCEPTED
-------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
-------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
-------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.

Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK

Subtotal:    2000 SEK
Discount:    200 SEK (10% loyalty discount for returning customers)
Total cost:  1800 SEK
-------------------------------------------
Estimated completion: 2026-05-23 21:00
===========================================

System returned the accepted repair order:
===========================================
           REPAIR ORDER #1
===========================================
Date:    2026-05-16 21:00
State:   ACCEPTED
-------------------------------------------
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
-------------------------------------------
Customer's problem description:
  The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
-------------------------------------------
Diagnostic report:
  The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.
```

```
Proposed repair tasks:
  - Replace battery temperature sensor: 1200 SEK
  - Replace front brake pads: 450 SEK
  - Full safety inspection: 350 SEK

Subtotal:    2000 SEK
Discount:    200 SEK (10% loyalty discount for returning customers)
Total cost:  1800 SEK
------------------------------------------
Estimated completion: 2026-05-23 21:00
==========================================



########################################################################



########################################################################

RECEPTIONIST: A walk-in customer gives phone 0999999999. Searching the registry.
Sorry, no customer was found for phone number 0999999999. Please double-check the number and
try again.

########################################################################



########################################################################

RECEPTIONIST: Searching for customer with phone 0700000000 (simulates an unreachable
database).
The customer database is temporarily unavailable. Please try again in a few minutes, or
contact IT support if the problem persists.

########################################################################



--- contents of repair-bike-error-log.txt after the run ---
2026-05-16 21:00:01  Database failure while searching for phone 0700000000
se.kth.iv1350.repairbike.integration.DatabaseFailureException: Could not reach the database
while looking up identifier 0700000000.
    at
se.kth.iv1350.repairbike.integration.CustomerRegistry.findCustomer(CustomerRegistry.java:48)
    at se.kth.iv1350.repairbike.controller.Controller.searchCustomer(Controller.java:71)
    at se.kth.iv1350.repairbike.view.View.trySearchCustomer(View.java:130)
    at se.kth.iv1350.repairbike.view.View.runDatabaseFailureFlow(View.java:115)
    at se.kth.iv1350.repairbike.view.View.sampleExecution(View.java:55)
    at se.kth.iv1350.repairbike.startup.Main.main(Main.java:25)
```

Figure 1: *Sample run output, captured from System.out, plus the contents of the error log produced by the run.*

4 Discussion

4.1 The nine exception-handling best practices

The list in section 8.2 of the textbook and in the seminar 4 task description is walked through below.

- Choose between checked and unchecked. CustomerNotFoundException is checked (business condition), DatabaseFailureException is unchecked (technical failure). Both choices are documented in the exception Javadoc and asserted by testCustomerNotFoundExceptionIsChecked and testDatabaseFailureExceptionIsUnchecked.
- Use the correct abstraction level. Neither exception leaks a lower-level type. DatabaseFailureException is named at the integration-layer abstraction and accepts a Throwable cause so the underlying technical exception can be chained without escaping to the higher layers.
- Name the exception after the error condition. Done for both.
- Include information about the error condition. CustomerNotFoundException.getMissingPhoneNumber() and DatabaseFailureException.getFailingIdentifier() carry the value that triggered the exception, in addition to a readable message.
- Use functionality provided in java.lang.Exception. Both call super(message), DatabaseFailureException additionally calls super(message, cause) to preserve the cause chain.
- Write Javadoc comments for all exceptions. Both classes have extensive Javadoc that also documents why one is checked and the other is unchecked.
- An object shall not change state if an exception is thrown. CustomerRegistry.findCustomer throws before any state change. RepairOrder.setDiscountStrategy validates its argument before touching the field. testSettingNullDiscountStrategyThrowsAndKeepsState verifies that the strategy is unchanged after a null argument is rejected.
- Notify users. The View prints distinct, non-technical messages on System.out for both exception paths. testSampleExecutionUserMessagesDoNotLeakTechnicalDetails verifies that no stack trace or exception class name leaks into the user-facing output.
- Notify developers. ErrorFileLogger writes a timestamped stack trace to repair-bike-error-log.txt only when the exception indicates the program is not functioning as intended (DatabaseFailureException). testCustomerNotFoundIsNotLogged verifies the business condition is not logged.
- Write unit tests for the exception handling. CustomerNotFoundExceptionTest, DatabaseFailureExceptionTest, CustomerRegistryTest, ControllerTest, ErrorFileLoggerTest, and ViewTest all cover parts of the exception handling.

4.2 Are the patterns correct implementations?

Observer: the Subject (RepairOrder) maintains a list of Observer references, allows add and remove, calls a single push-based update method on every observer when its state changes, and defensively iterates over a snapshot of the list. The data pushed is an immutable DTO. This matches the description of the pattern. The Subject does not know the concrete classes of the observers, only the RepairOrderObserver interface.

Singleton: the constructor is private, the instance is held in a private static volatile field, and getInstance() uses double-checked locking. No subclass can break the invariant because the

constructor is inaccessible. createForTests() is a deliberate, documented escape hatch for testability, separate from getInstance().

Strategy: a clean strategy interface (DiscountStrategy) with two methods, three concrete implementations, and a context (RepairOrder) that holds a reference and delegates the algorithm. The strategy is replaceable at runtime through setDiscountStrategy and notifications fire so observers can pick up the change. The strategy is also testable in isolation: DiscountStrategyTest covers each implementation directly without going through the RepairOrder.

4.3 Where is the observer reference passed?

Observers are created in Main and added to the Controller, which then registers them on every RepairOrder it creates. The cost of this approach is that the Controller holds a list of observer references it does not itself use. The benefit is that the wiring exists in exactly one place (Main) and the Controller only knows the observer interface, not any concrete implementation. Coupling stays low (no concrete observer is named in the controller code) and cohesion is preserved (Controller still does the same thing as before, plus the small bookkeeping of attaching observers to new orders).

4.4 What data is passed to observers?

Observers receive an immutable RepairOrderDTO that carries the repair-order id, state, creation time, discount fields, an unmodifiable list of proposed repair tasks, the estimated completion time, and a pre-formatted printout. None of these fields exposes a reference to the live RepairOrder entity. The DTO is the same type returned by the Controller, which keeps the model's encapsulation intact. The chosen field set is the smallest one that satisfies what RepairOrderView and RepairOrderLogger actually need. If a future observer needs more, the addition would happen in one place (RepairOrderDTO + RepairOrder.toDTO()).

4.5 Trade-offs and possible improvements

The Singleton design is not without drawbacks. The createForTests() factory is a deliberate compromise: it preserves the Singleton guarantee in production while giving tests the isolation they need. A more invasive alternative would be to abandon the Singleton in favour of passing the registries explicitly through every constructor, which would remove the testability concern at the cost of a longer constructor signature.

References

Lindback, L. (2026). A First Course in Object-Oriented Development

AI usage

ChatGPT 5.5 was used to improve spelling and grammar in this report.